Programming Techniques 2026–2027

Lecture 12: Introduction to MPI

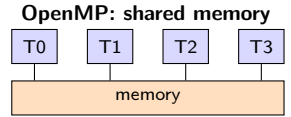
Hannu Parviainen

Universidad de la Laguna

November 9, 2026

From OpenMP to MPI

- **OpenMP** (Lectures 9 and 10): the threads of one program share the memory of one computer.
- **MPI**: many copies of the program run as separate *processes*. Each process has its own memory, with its own copy of every variable.
- A process cannot see the variables of another one. The processes exchange data only by sending and receiving *messages*.
- The processes can run on one computer, or on many nodes of a cluster.



	OpenMP	MPI
runs as	threads	processes
memory	shared	distributed
scale	one node	many nodes

What is MPI?

- The *Message Passing Interface* is a standard for a library of message-passing routines, with bindings for Fortran and C. The first version, MPI-1, was published in 1994, and the current one is MPI-5.0 (2025).
- The standard is implemented by several libraries: **Open MPI** and **MPICH** are free, and vendors such as Intel and HPE ship their own. A correct program works with all of them.
- MPI is the standard way to run programs on supercomputers.
- Usually every process runs the same program, and works on its own part of the data. This is called SPMD: *single program, multiple data*.
- The processes are grouped into *communicators*. `MPI_COMM_WORLD` contains all the processes.
- In a communicator of p processes, each process has a number, its *rank*, from 0 to $p - 1$. The program uses the rank to decide what each process does.

Getting Started

Installing MPI

- **Conda**: `openmpi-mpifort` installs Open MPI with `gfortran`.
- **Ubuntu**: `sudo apt install openmpi-bin libopenmpi-dev`
- **macOS**: `brew install open-mpi`
- **Windows**: use Linux under WSL.
- **Clusters**: use the MPI of the cluster, e.g. `module load openmpi`.

`mpifort` and `mpirun` must come from the same MPI installation.

```
$ conda create -n mpi -c conda-forge openmpi-mpifort
$ conda activate mpi
$ mpifort --version
GNU Fortran (conda-forge gcc 16.2.0-7) 16.2.0
$ mpirun --version
mpirun (Open MPI) 5.0.11
```

`mpifort` is a *wrapper*: it calls `gfortran` with the paths to the MPI modules and libraries. `mpifort --showme` (Open MPI) or `mpifort -show` (MPICH) prints the full command.

On macOS, the linker may warn about a duplicate `-rpath`. The warning is harmless.

Hello, MPI

- `use mpi_f08` gives the MPI procedures, types, and constants.
- `mpi_init` starts MPI. It must be called before any other MPI routine.
- `mpi_comm_size` gives the number of processes, and `mpi_comm_rank` the rank of this process.
- `mpi_finalize` shuts MPI down. Every rank must call it before the program ends.
- Every rank runs the whole program. Only the value of `rank` differs.

`hello.f90`:

```
program hello
  use mpi_f08
  implicit none
  integer :: rank, n_ranks

  call mpi_init()
  call mpi_comm_size(MPI_COMM_WORLD, n_ranks)
  call mpi_comm_rank(MPI_COMM_WORLD, rank)

  print *, 'Hello from rank', rank, 'of', n_ranks

  call mpi_finalize()
end program hello
```

Do not call a variable `size`: it would hide the intrinsic function `size()`.

Compiling and running

```
$ mpifort -Wall -o hello hello.f90
$ mpirun -np 4 ./hello
 Hello from rank           0 of           4
 Hello from rank           2 of           4
 Hello from rank           3 of           4
 Hello from rank           1 of           4
```

- `mpirun -np 4` starts four copies of the program, and connects them into one `MPI_COMM_WORLD`.
- `mpiexec` is the name in the MPI standard. `mpirun` does the same.

- The ranks run independently, so their lines come in a random order.
- Open MPI refuses to start more ranks than there are cores. To test with more:

```
$ mpirun --oversubscribe -np 16 ./hello
```

  The run times of an oversubscribed run are meaningless.
- On a cluster, the batch system (e.g. Slurm) decides where the ranks run.
- The `makefile` in `lectures/l12` builds all the examples.

The mpi_f08 module

- MPI has three Fortran interfaces: `use mpi_f08` (modern, recommended), `use mpi` (older, common in existing codes), and `include 'mpif.h'` (deprecated).
- In `mpi_f08`, the handles are derived types, such as `type(MPI_Comm)` and `type(MPI_Status)`, so the compiler checks the arguments.
- The last argument, `ierror`, is optional.
- **Errors.** By default, an error in an MPI call aborts the whole program, so checking `ierror` is rarely useful. To stop all the ranks yourself, call `mpi_abort`. A `stop` on one rank can leave the others waiting.

With `use mpi` (older codes and books):

```
integer :: ierr, status(MPI_STATUS_SIZE)
call mpi_recv(x, 1, MPI_REAL, 0, 0, &
              MPI_COMM_WORLD, status, ierr)
print *, status(MPI_SOURCE)
```

With `use mpi_f08`:

```
type(MPI_Status) :: status
call mpi_recv(x, 1, MPI_REAL, 0, 0, &
              MPI_COMM_WORLD, status)
print *, status%MPI_SOURCE
```

Stopping all the ranks with the error code 1:

```
call mpi_abort(MPI_COMM_WORLD, 1)
```

MPI datatypes

- The data of a message is given by three arguments: the *buffer*, a variable or an array, the *count*, the number of elements, and the *datatype* of the elements.
- The count is in elements, not in bytes.
- The datatype must match the Fortran type of the buffer. The compiler cannot check this: a `real(dp)` array sent as `MPI_REAL` gives wrong numbers without an error.
- Send whole arrays or contiguous sections. Fortran stores arrays by column, so `a(:,j)` is contiguous, but `a(i,:)` is not.

Fortran type	MPI datatype
`integer`	`MPI_INTEGER`
`real`	`MPI_REAL`
`real(real64)`	`MPI_DOUBLE_PRECISION`
`complex`	`MPI_COMPLEX`
`complex(real64)`	`MPI_DOUBLE_COMPLEX`
`logical`	`MPI_LOGICAL`
`character`	`MPI_CHARACTER`

In this course, `dp = real64`, so all the `real(dp)` data is sent as `MPI_DOUBLE_PRECISION`.

Point-to-Point Communication

mpi_send and mpi_recv

```
call mpi_send(buf, count, datatype, dest, tag, comm)
call mpi_recv(buf, count, datatype, source, tag, comm, status)
```

- A message goes from one rank to one other rank. Besides the data, it carries an *envelope*: the source, the destination, a *tag*, and the communicator.
- A receive takes a message whose envelope matches its `source`, `tag`, and `comm`.
- The tag is an integer ≥ 0 that the program chooses, to tell different kinds of messages apart.
- The count of `mpi_recv` is the size of the buffer. A longer message is an error.
- Both calls are *blocking*: `mpi_send` returns when `buf` can be reused, and `mpi_recv` when the message has arrived in `buf`.
- Messages from one rank to another do not overtake each other: they are received in the order they were sent. There is no order between messages from different ranks.
- `status` tells the source and the tag of the message. Pass `MPI_STATUS_IGNORE` if you do not need it.

Example: sending an array

- In `send_recv.f90`, rank 0 sends four numbers to rank 1, with the tag 42.
- Rank 1 receives them into its own `x`: the same name, but a different variable.
- With one rank only, `mpi_send` fails: rank 1 does not exist.

```
$ mpirun -np 2 ./send_recv
 Rank 1 received   1.0000000000000000
   4.0000000000000000    9.0000000000000000
   16.000000000000000
```

```
program send_recv
  use mpi_f08
  use iso_fortran_env, only: dp => real64
  implicit none
  integer :: rank
  real(dp) :: x(4)

  call mpi_init()
  call mpi_comm_rank(MPI_COMM_WORLD, rank)

  if (rank == 0) then
    x = [1.0_dp, 4.0_dp, 9.0_dp, 16.0_dp]
    call mpi_send(x, 4, MPI_DOUBLE_PRECISION, 1, 42, &
                  MPI_COMM_WORLD)
  else if (rank == 1) then
    call mpi_recv(x, 4, MPI_DOUBLE_PRECISION, 0, 42, &
                  MPI_COMM_WORLD, MPI_STATUS_IGNORE)
    print *, 'Rank 1 received', x
  end if

  call mpi_finalize()
end program send_recv
```

Receiving from any rank

- In `any_source.f90`, every worker rank sends a result to rank 0.
- Rank 0 does not know which worker finishes first. With `MPI_ANY_SOURCE` and `MPI_ANY_TAG`, it takes the messages in the order they arrive.
- `status%MPI_SOURCE` tells who sent each message.

```
$ mpirun -np 4 ./any_source
 Got           1 from rank           1
 Got           9 from rank           3
 Got           4 from rank           2
```

```
program any_source
  use mpi_f08
  implicit none
  integer :: rank, n_ranks, i, x
  type(MPI_Status) :: status

  call mpi_init()
  call mpi_comm_rank(MPI_COMM_WORLD, rank)
  call mpi_comm_size(MPI_COMM_WORLD, n_ranks)

  if (rank == 0) then
    do i = 1, n_ranks - 1
      call mpi_recv(x, 1, MPI_INTEGER, MPI_ANY_SOURCE, &
                    MPI_ANY_TAG, MPI_COMM_WORLD, status)
      print *, 'Got', x, 'from rank', status%MPI_SOURCE
    end do
  else
    x = rank**2
    call mpi_send(x, 1, MPI_INTEGER, 0, 0, MPI_COMM_WORLD)
  end if

  call mpi_finalize()
end program any_source
```

Deadlock

Two ranks swap arrays of n numbers. Both send first, and then receive (`exchange.f90`):

```
call mpi_send(a, n, MPI_DOUBLE_PRECISION, other, 0, &
              MPI_COMM_WORLD)
call mpi_recv(b, n, MPI_DOUBLE_PRECISION, other, 0, &
              MPI_COMM_WORLD, MPI_STATUS_IGNORE)
```

- MPI may copy a small message into a buffer, and return from `mpi_send` at once. A large message waits until the other rank receives it.
- Then both ranks wait in `mpi_send` for a receive that never comes: a *deadlock*.
- With Open MPI 5 on a laptop, $n = 500$ (4 kB) works, but $n = 1000$ (8 kB) hangs. The limit depends on the MPI library and the network.

A correct program must not depend on buffering.

Fix 1: order the calls so that every send meets a receive:

```
if (rank == 0) then
  call mpi_send(a, ...)
  call mpi_recv(b, ...)
else
  call mpi_recv(b, ...)
  call mpi_send(a, ...)
end if
```

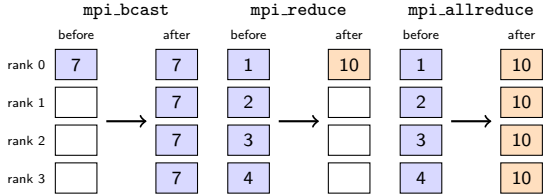
Fix 2: send and receive at once:

```
call mpi_sendrecv( &
    a, n, MPI_DOUBLE_PRECISION, other, 0, &
    b, n, MPI_DOUBLE_PRECISION, other, 0, &
    MPI_COMM_WORLD, MPI_STATUS_IGNORE)
```

Collective Communication

Collective operations

- A *collective* operation involves all the ranks of a communicator, e.g. to send data to all of them, or to combine data from all of them.
- *Every* rank must call it, with matching arguments. If one rank skips the call, the others wait forever.
- Many collectives have a *root*: the rank that sends or receives the data. They have no tags.
- A collective is faster than many sends: MPI passes the data along a tree, in about $\log_2 p$ steps.



`mpi_bcast` copies the data of the root to every rank. `mpi_reduce` combines the data of all the ranks, here with `MPI_SUM`, into the root. `mpi_allreduce` gives the result to every rank. Here, the root is rank 0.

Broadcast

```
call mpi_bcast(buffer, count, datatype, root, comm)
```

- On the root, `buffer` holds the data to send. On the other ranks, it receives the data.
- A common use: rank 0 reads the input, and broadcasts it to the others. With `mpirun`, only rank 0 can read the standard input.

```
$ echo 1000 | mpirun -np 4 ./bcast
 Rank           0 has n =        1000
 Rank           2 has n =        1000
 Rank           3 has n =        1000
 Rank           1 has n =        1000
```

`bcast.f90`:

```
program bcast
  use mpi_f08
  implicit none
  integer :: rank, n

  call mpi_init()
  call mpi_comm_rank(MPI_COMM_WORLD, rank)

  if (rank == 0) read *, n     ! only rank 0 reads
  call mpi_bcast(n, 1, MPI_INTEGER, 0, MPI_COMM_WORLD)
  print *, 'Rank', rank, 'has n =', n

  call mpi_finalize()
end program bcast
```

Reduce and allreduce

```
call mpi_reduce(sendbuf, recvbuf, count, datatype, op, root, comm)
call mpi_allreduce(sendbuf, recvbuf, count, datatype, op, comm)
```

- The operation op combines the values of all the ranks: `MPI_SUM`, `MPI_PROD`, `MPI_MAX`, `MPI_MIN`, ...
- Arrays are combined element by element.
- `sendbuf` and `recvbuf` must be different variables. To combine into the same array, pass `MPI_IN_PLACE` as `sendbuf`.
- This is the MPI version of the OpenMP `reduction` clause.

The total energy, on rank 0 only:

```
call mpi_reduce(e, e_tot, 1, MPI_DOUBLE_PRECISION, &
                MPI_SUM, 0, MPI_COMM_WORLD)
```

The largest speed, on every rank:

```
call mpi_allreduce(v, v_max, 1, MPI_DOUBLE_PRECISION, &
                   MPI_MAX, MPI_COMM_WORLD)
```

A histogram summed over all the ranks, in place:

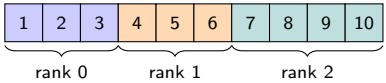
```
call mpi_allreduce(MPI_IN_PLACE, counts, n_bins, &
                   MPI_INTEGER, MPI_SUM, MPI_COMM_WORLD)
```

Dividing the work between the ranks

- To share a loop over n items between p ranks, each rank computes its own block from its rank. No communication is needed.
- If p does not divide n, the blocks differ in size by at most one:

```
i1 = rank * n / n_ranks + 1
i2 = (rank + 1) * n / n_ranks
do i = i1, i2
  ...
end do
```

The integer division rounds down, so the blocks join without gaps or overlaps.



$n = 10$ items and $p = 3$ ranks.

If the items take different amounts of work, a *cyclic* division balances the load better, like `schedule(static, 1)` in OpenMP:

```
do i = rank + 1, n, n_ranks
  ...
end do
```

Example: the trapezoid rule

With n trapezoids of width $h = (b - a)/n$,

$$\int_a^b f(x)\,\mathrm{d}x \approx h\left[\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + ih)\right]$$

- Each rank sums f over its own block of the inner points $i = 1, \ldots, n-1$.
- `mpi_reduce` adds the partial sums on rank 0.
- The end points must be added only once in total, so only rank 0 adds them.
- With $f(x) = 4/(1 + x^2)$ on $[0, 1]$, the integral is π.

Ranks	**Result** ($n = 10^6$)
1	3.1415926535897927
2	3.1415926535895884
3	3.1415926535895680
4	3.1415926535895973
8	3.1415926535896328
π	3.1415926535897932

The last digits change with the number of ranks, because the partial sums are added in a different order. Floating-point addition is not associative.

The trapezoid rule with MPI

```
program trapezoid
  use mpi_f08
  use iso_fortran_env, only: dp => real64
  implicit none
  integer, parameter :: n = 1000000
  real(dp), parameter :: a = 0.0_dp, b = 1.0_dp
  integer :: rank, n_ranks, i, i1, i2
  real(dp) :: h, local, total

  call mpi_init()
  call mpi_comm_rank(MPI_COMM_WORLD, rank)
  call mpi_comm_size(MPI_COMM_WORLD, n_ranks)

  ! This rank's block of the inner points
  h = (b - a) / n
  i1 = rank * (n - 1) / n_ranks + 1
  i2 = (rank + 1) * (n - 1) / n_ranks

  local = 0.0_dp
  do i = i1, i2
    local = local + f(a + i * h)
  end do

  ! Add the end points only once in total
  if (rank == 0) then
    local = local + 0.5_dp * (f(a) + f(b))
  end if

  call mpi_reduce(local, total, 1, &
                  MPI_DOUBLE_PRECISION, MPI_SUM, &
                  0, MPI_COMM_WORLD)
  if (rank == 0) print *, 'Integral:', h * total

  call mpi_finalize()

contains

  pure real(dp) function f(x)
    real(dp), intent(in) :: x
    f = 4.0_dp / (1.0_dp + x**2)
  end function f

end program trapezoid
```

Timing

- `mpi_wtime()` returns the wall-clock time in seconds as a `real(real64)`, like `omp_get_wtime()`.
- Each rank has its own clock, so use only time differences.
- A parallel program is as fast as its slowest rank. Start the ranks together with `mpi_barrier`, which waits until all the ranks reach it, and report the largest time.
- Amdahl's law (Lecture 10) holds for MPI too. The time spent in communication adds to the serial part.

```
real(dp) :: t0, t, t_max

call mpi_barrier(MPI_COMM_WORLD)
t0 = mpi_wtime()

! ... the work ...

t = mpi_wtime() - t0
call mpi_reduce(t, t_max, 1, MPI_DOUBLE_PRECISION, &
                MPI_MAX, 0, MPI_COMM_WORLD)
if (rank == 0) print *, 'Time (s):', t_max
```

Common pitfalls

- **Work done once per rank.** Work that must be done once in total, such as output or the end points of a sum, needs an `if (rank == 0)`.
- **A wrong datatype or count.** The compiler cannot check them, and the result is wrong numbers, not an error.
- **Deadlock.** Two ranks that both wait for each other.
- **A collective called by only some ranks.** The others wait forever.
- **Output from many ranks.** The lines come in a random order. Print from rank 0.
- **Mixed MPI installations.** If `mpirun` is not from the same MPI as `mpifort`, every process says it is "rank 0 of 1". Check with `which mpifort mpirun`.
- **Testing with one rank only.** Compare the results with 1, 2, 3, and 4 ranks. An odd number of ranks finds bugs in the division of the work.
- **A hang.** Find the call in which each rank waits.

Summary

Routine	Meaning
`mpi_init`, `mpi_finalize`	start and end MPI
`mpi_comm_size`	number of ranks
`mpi_comm_rank`	rank of this process
`mpi_send`, `mpi_recv`	one message, blocking
`mpi_sendrecv`	send and receive at once
`mpi_bcast`	root's data to all
`mpi_reduce`	combine data into the root
`mpi_allreduce`	combine data into all
`mpi_barrier`	wait for all the ranks
`mpi_wtime`	wall-clock time
`mpi_abort`	stop all the ranks

Compile with `mpifort`, and run with `mpirun -np N`.

- The MPI standard: `https://www.mpi-forum.org/docs/`
- Open MPI documentation: `https://docs.open-mpi.org/`
- Books:
  - W. Gropp, E. Lusk, A. Skjellum, *Using MPI* (3rd ed., MIT Press, 2014),
  - P.S. Pacheco, *Parallel Programming with MPI* (Morgan Kaufmann, 1997).

  Both use the older interfaces, but the routines are the same.

Exercise 1: a ring

Tasks

1. Arrange the ranks in a ring: rank r sends to rank $(r+1) \bmod p$, and receives from rank $(r-1+p) \bmod p$.
2. Pass a token around the ring: rank 0 sends the value 0 to rank 1, every rank adds its rank to the value and passes it on, and rank 0 receives the final value from rank $p-1$ and prints it. Check the result against $p(p-1)/2$.
3. Now let every rank send an array of n numbers to its right neighbour, and receive one from its left neighbour, at the same time, with `mpi_send` followed by `mpi_recv`. Increase n until the program deadlocks. Fix it with `mpi_sendrecv`.

Exercise 2: Monte Carlo π

Tasks

1. Write a program that draws N random points in the unit square, and counts how many fall inside the quarter circle $x^2 + y^2 < 1$. The fraction inside approaches $\pi/4$.
2. Share the N points between the ranks, and combine the counts with `mpi_reduce`. Rank 0 reads N from the standard input, and broadcasts it.
3. Give every rank a different seed with `random_seed(put=...)`. What goes wrong if all the ranks use the same seed?
4. Time the program with `mpi_wtime` for $N = 10^8$ and 1, 2, 4, and 8 ranks, and compute the speedups.

Exercise 3: the trapezoid rule

Tasks

1. Change `trapezoid.f90` so that rank 0 reads a, b, and n from the standard input, and broadcasts them to the other ranks.
2. Check that the result does not depend on the number of ranks, for n divisible and not divisible by the number of ranks.
3. Time the program with `mpi_wtime` for $n = 10^8$ with 1, 2, 4, and 8 ranks, and compare the speedups with Amdahl's law.
4. With $n = 10^9$ and four ranks, the program gives a wrong result. Why? Fix it. (Hint: `huge(1)`.)